

IE-GB Interconnector Flow Prediction Model (XGBoost)

Net interconnector flow is primarily driven by the ISEM/GB day-ahead price spread, with wind generation imbalances and system demand acting as secondary drivers.

This model uses XGBoost to forecast net IC flow at 30-minute resolution for the following delivery day. It is designed to be run around or after 14:00 on D-1, once both the EPEX GB (approx. 09:00) and SEMOpx (approx. 12:00) day-ahead auctions have cleared and their results have been loaded to our database.

Features combine day-ahead market prices, wind and demand forecasts, derived spread and position metrics, and lagged flow history. Hyperparameters are tuned via Optuna with time-series cross-validation to prevent lookahead bias.

Import libraries and functions:

```
In [1]: import pandas as pd
import numpy as np
import plotly.graph_objects as go
import xgboost as xgb
from xgboost import XGBRegressor
import optuna
from sklearn.model_selection import train_test_split
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from datetime import datetime as dt
from Common.DBconnect import dbConnect

# Define start and end dates:
start_date = '2025-04-18 23:00'
end_date = pd.Timestamp.today().normalize() + pd.Timedelta(days=2)
```

MySQL Database Query Functions:

Two helper functions are used to pull data from a database that stores market data.

`db_query` fetches a single time series for a given ID between a start and end date. Where multiple forecast versions exist for the same timestamp, the median value is taken.

`get_plant_outages` fetches the latest published outage records for the three IE-GB interconnectors - EWIC, Moyle, and Greenlink. It returns the most recently modified version of each published outage that overlaps the requested date window, along with installed capacity mapped from a lookup dictionary.

```
In [2]: # Fetches a single time series from the database for a particular ID.
def db_query(start_date, end_date, dim_id, quantity_label):
    query = """
    SELECT
        ts.TimestampUTC AS start_time_utc,
        ts.Value AS quantity
    FROM DataWarehouse.TimeSeries ts
    WHERE ts.SeriesID = '{0}'
        AND ts.TimestampUTC BETWEEN '{1}' AND '{2}'
    ORDER BY start_time_utc
    """.format(dim_id, start_date, end_date)

    df = pd.DataFrame(dbConnect.select(query, "DataWarehouse"),
                      columns=['Time (UTC)', quantity_label])
    df['Time (UTC)'] = pd.to_datetime(df['Time (UTC)'])

    # Cast values to float (SQL DECIMAL types are returned as Python decimal.Decimal)
    df[quantity_label] = pd.to_numeric(df[quantity_label], errors="coerce").astype(float)

    df = df.set_index('Time (UTC)')

    # Due to multiple updated forecast ticks, we take the median across all forecast versions for each timestamp
    df = df.groupby(level=0).median()

    return df

# Fetches the Latest published outage records for selected transmission assets.
def get_plant_outages(start_date, end_date):
    query = """
    SELECT assets.AssetName,
           outages.AvailableCapacity,
           outages.OutageStart,
           outages.OutageEnd
    FROM (
        -- Get the latest published version of each outage
        SELECT AssetID,
               PublicationDate,
               MAX(RevisionDate) RevisionDate
        FROM DataWarehouse.AssetOutages
        WHERE RecordType = 4
        GROUP BY AssetID, PublicationDate
    ) latest
    JOIN DataWarehouse.vw_AssetOutages outages
    USING (AssetID, PublicationDate, RevisionDate)
    JOIN DataWarehouse.vw_Assets assets
    ON assets.AssetID = outages.AssetID
    WHERE assets.AssetName IN ('INTERCONNECTOR_A',
                              'INTERCONNECTOR_B',
                              'INTERCONNECTOR_C')
        -- Include any outage that overlaps the requested window
    AND ((OutageStart >= '{0}' AND OutageStart < '{1}')
        OR (OutageEnd >= '{0}' AND OutageEnd < '{1}'))
        OR (OutageStart < '{0}' AND OutageEnd >= '{0}'))
    AND AssetStatus = 'Active'
    """
```

```
        AND (RetirementDate IS NULL OR RetirementDate > '{0}'))
        ORDER BY AssetName, OutageStart, OutageEnd, OutageID;
        """,format(start_date, end_date)

df = pd.DataFrame(
    dbConnect.select(query, "DataWarehouse"),
    columns=['Asset Name', 'Available (MW)', 'Start (UTC)', 'End (UTC)']
)

df['Installed Capacity (MW)'] = df['Asset Name'].map(installed_capacities)
df['Available (MW)'] = pd.to_numeric(df['Available (MW)'], errors='coerce')
df['Start (UTC)'] = pd.to_datetime(df['Start (UTC)'])
df['End (UTC)'] = pd.to_datetime(df['End (UTC)'])

return df[['Asset Name', 'Installed Capacity (MW)', 'Available (MW)', 'Start (UTC)', 'End (UTC)']]
```

Feature Data

Forecast data is pulled for wind, solar, demand and day-ahead prices across both IE and GB. GB prices are fetched in GBP and converted to EUR using a daily FX rate that is forward-filled to 30-minute resolution.

Actual flows are fetched for each of the three interconnectors (EWIC, Moyle, Greenlink) and flipped negative to align with the IE export convention, where positive values represent flow from GB to IE.

Plant outages are fetched for each interconnector using the installed capacities below.

Interconnector	Installed Capacity (MW)
EWIC	532
Moyle	452
Greenlink	514

```
In [3]: forecast_end = pd.Timestamp(end_date) + pd.Timedelta(days=1)

IE_wind_forecast = db_query(start_date, forecast_end, 9352, "IE Wind Forecast (MW)")
IE_solar_forecast = db_query(start_date, forecast_end, 17571, "IE Solar Forecast (MW)")
GB_wind_forecast = db_query(start_date, forecast_end, 4485, "GB Wind Forecast (MW)")
GB_solar_forecast = db_query(start_date, forecast_end, 4493, "GB Solar Forecast (MW)")
IE_demand_forecast = db_query(start_date, forecast_end, 9348, "IE Demand Forecast (MW)")
GB_demand_forecast = db_query(start_date, forecast_end, 3995, "GB Demand Forecast (MW)")

price_start = pd.Timestamp(start_date) - pd.Timedelta(days=1)

IE_DA_prices_EUR = db_query(price_start, forecast_end, 9408, "IE DA Prices (€)")
GB_DA_IMRP_prices_GBP = db_query(price_start, forecast_end, 18378, "GB DA Prices (£)")
IE_DA_prices_EUR = db_query(start_date, forecast_end, 9408, "IE DA Prices (€)")
GB_DA_IMRP_prices_GBP = db_query(start_date, forecast_end, 18378, "GB DA Prices (£)")

def get_GBP_EUR_fx_rate(start_date, end_date):
    adjusted_start = (pd.to_datetime(start_date) - pd.Timedelta(days=1)).strftime('%Y-%m-%d')

    query = """
    SELECT
        trade_date,
        exchange_rate
    FROM MarketData.fx_exchange_rates
    WHERE trade_date BETWEEN '{0}' AND '{1}'
        AND (base_currency = 'GBP' AND quote_currency = 'EUR')
    """,format(adjusted_start, end_date)

    df = pd.DataFrame(
        dbConnect.select(query, "MarketData"),
        columns=['Time (UTC)', 'GBP-EUR FX Rate (€)']
    )

    df['Time (UTC)'] = pd.to_datetime(df['Time (UTC)'])
    df['GBP-EUR FX Rate (€)'] = df['GBP-EUR FX Rate (€)'].astype(float)

    df = df.drop_duplicates(subset='Time (UTC)')
    df = df.set_index('Time (UTC)')
    df = df.resample('30 min').ffill()
    df = df.astype(float)

    return df

GBP_EUR_fx_rate = get_GBP_EUR_fx_rate(start_date, end_date)

def get_GB_DA_prices_EUR(start_date, end_date):
    df = GB_DA_IMRP_prices_GBP.copy()
    df = df.astype(float)
    df['GB DA Prices (€)'] = df['GB DA Prices (£)'] * GBP_EUR_fx_rate['GBP-EUR FX Rate (€)']
    return df[['GB DA Prices (€)']]

GB_DA_prices_EUR = get_GB_DA_prices_EUR(start_date, end_date)

# Shift DA prices forward by 24h to align trade date with delivery date
IE_DA_prices_EUR.index = IE_DA_prices_EUR.index + pd.Timedelta(days=1)
GB_DA_prices_EUR.index = GB_DA_prices_EUR.index + pd.Timedelta(days=1)

ewic_flow_actual = -db_query(start_date, end_date, 4871, 'EWIC Actual Flow (MW)')
moyle_flow_actual = -db_query(start_date, end_date, 4872, 'Moyle Actual Flow (MW)')
greenlink_flow_actual = -db_query(start_date, end_date, 18349, 'Greenlink Actual Flow (MW)')

installed_capacities = {
    "IC_MOYLE" : 452.0,
    "IC_GREENLINK" : 514.0,
```

```
    "IC_EWIC" : 532.0
}

plant_outages = get_plant_outages(start_date, end_date)
```

Feature Engineering

All features are concatenated into a single dataframe at 30-minute resolution. Hourly forecast series are forward-filled to align with the half-hourly target. Day-ahead price series for Ireland and Great Britain are shifted forward by 24 hours so that the auction date aligns with the corresponding delivery day.

The target variable is the sum of actual flows across the three interconnectors.

Several derived features are calculated from the forecast and price data:

- **Net position** for IE and GB: demand minus wind and solar forecast, used as a proxy for how much each market needs to import or can afford to export
- **Wind penetration** for IE and GB: wind forecast as a share of demand forecast
- **Day-ahead price spread**: GB day-ahead price minus IE day-ahead price, used to capture the market incentive for cross-border flow
- **Interconnector capacity**: starts at installed capacity, with outage records applied on top for EWIC, Moyle, and Greenlink

Rows where the target or key price features are missing are dropped. This removes future timestamps where no actual flow data exists yet, as well as any timestamps where shifted day-ahead prices are unavailable.

In [4]:

```
# Create a concatenated dataframe of all features
model_df = pd.concat([
    IE_wind_forecast,
    GB_wind_forecast,
    IE_solar_forecast,
    GB_solar_forecast,
    IE_demand_forecast,
    GB_demand_forecast,
    ewic_flow_actual,
    moyle_flow_actual,
    greenlink_flow_actual,
    IE_DA_prices_EUR,
    GB_DA_prices_EUR
], axis=1)

model_df.index = pd.to_datetime(model_df.index)
model_df = model_df.sort_index()

# Forward fill hourly forecasts to 30-min resolution
forecast_cols = [
    "IE Wind Forecast (MW)",
    "GB Wind Forecast (MW)",
    "IE Solar Forecast (MW)",
    "GB Solar Forecast (MW)",
    "IE Demand Forecast (MW)",
    "GB Demand Forecast (MW)",
]
model_df[forecast_cols] = model_df[forecast_cols].ffill()

# Target
model_df["Net IC Flows (MW)"] = model_df[[
    "EWIC Actual Flow (MW)",
    "Greenlink Actual Flow (MW)",
    "Moyle Actual Flow (MW)",
]].sum(axis=1)

# IE-GB Price Spread
model_df["DA Price Spread (€)"] = (
    model_df["GB DA Prices (€)"] - model_df["IE DA Prices (€)"]
)

# Forecast-derived features - no Leakage, all from forecasts
model_df["GB Net Position (MW)"] = (
    model_df["GB Demand Forecast (MW)"]
    - model_df["GB Wind Forecast (MW)"]
    - model_df["GB Solar Forecast (MW)"]
)
model_df["IE Net Position (MW)"] = (
    model_df["IE Demand Forecast (MW)"]
    - model_df["IE Wind Forecast (MW)"]
    - model_df["IE Solar Forecast (MW)"]
)
model_df["IE Wind Penetration"] = model_df["IE Wind Forecast (MW)"] / model_df["IE Demand Forecast (MW)"]
model_df["GB Wind Penetration"] = model_df["GB Wind Forecast (MW)"] / model_df["GB Demand Forecast (MW)"]

# Interconnector capacities - start at installed capacity then apply outages
plant_col_map = {
    "IC_EWIC": "EWIC Capacity (MW)",
    "IC_GREENLINK": "Greenlink Capacity (MW)",
    "IC_MOYLE": "Moyle Capacity (MW)",
}
for plant, col in plant_col_map.items():
    model_df[col] = installed_capacities[plant]

for _, row in plant_outages.iterrows():
    col = plant_col_map.get(row["Plant Name"])
    if col:
        mask = (model_df.index >= row["Start (UTC)"]) & (model_df.index <= row["End (UTC)"])
        model_df.loc[mask, col] = row["Available (MW)"]

model_df["Total IC Capacity (MW)"] = (
    model_df["EWIC Capacity (MW)"]
    + model_df["Moyle Capacity (MW)"]
    + model_df["Greenlink Capacity (MW)"]
)
```

```
# Drop rows where target is NaN (future rows with no actual flow data)
model_df = model_df.dropna(subset=["Net IC Flows (MW)"])

print(f"Rows after cleaning: {len(model_df)}")
print(f"Date range: {model_df.index[0]} to {model_df.index[-1]}")
display(model_df.head())
```

Rows after cleaning: 22212
Date range: 2025-04-18 23:00:00 to 2026-07-27 00:00:00

	IE Wind Forecast (MW)	GB Wind Forecast (MW)	IE Solar Forecast (MW)	GB Solar Forecast (MW)	IE Demand Forecast (MW)	GB Demand Forecast (MW)	EWIC Actual Flow (MW)	Moyle Actual Flow (MW)	Greenlink Actual Flow (MW)	IE DA Prices (€)	...	Net IC Flows (MW)	DA Price Spread (€)	GB Net Position (MW)	IE Net Position (MW)	IE Wind Penetration	GB Wind Penetration	C
Time (UTC)																		
2025-04-18 23:00:00	1841.0	10017.5	0.0	0.0	4088.0	20736.0	-0.0	370.0	NaN	NaN	...	370.0	NaN	10718.5	2247.0	0.450342	0.483097	
2025-04-18 23:30:00	1841.0	10017.5	0.0	0.0	4088.0	20736.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	10718.5	2247.0	0.450342	0.483097	
2025-04-19 00:00:00	1624.0	10761.0	0.0	0.0	3938.0	20422.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	9661.0	2314.0	0.412392	0.526932	
2025-04-19 00:30:00	1624.0	10761.0	0.0	0.0	3938.0	20422.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	9661.0	2314.0	0.412392	0.526932	
2025-04-19 01:00:00	1546.0	10262.0	0.0	0.0	3876.0	19995.0	-0.0	350.0	514.0	NaN	...	864.0	NaN	9733.0	2330.0	0.398865	0.513228	

5 rows × 21 columns



XGBoost Regression Model:

XGBoost (Extreme Gradient Boosting) is an ensemble method that builds a sequence of decision trees, where each tree is trained to correct the residual errors of the previous. The final prediction is the sum of all trees:

$$\hat{y}_i = \sum_{k=1}^K f_k(x_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F}$ is the space of regression trees and K is the total number of trees. At each boosting round, a new tree f_t is added to minimise a regularised objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

where l is a differentiable loss function (RMSE here) and Ω penalises model complexity:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j|$$

T is the number of leaves, w_j are the leaf weights, and γ, λ, α control the degree of regularisation. The objective is approximated using a second-order Taylor expansion, allowing efficient closed-form computation of the optimal leaf weights.

XGBoost suits this problem for several reasons. Interconnector flow is a non-linear function of multiple interacting variables - price spreads, wind generation and demand do not act independently, and their relationships shift across seasons and market conditions. Tree-based methods capture these interactions naturally without requiring explicit feature engineering of cross terms. The regularisation terms guard against overfitting on the relatively noisy half-hourly flow data, and the model handles the mix of continuous forecast inputs and cyclically encoded time features without requiring normalisation.

```
In [5]: # Create a copy to protect the original dataframe
xgboost_df = model_df.copy()

# Force all data columns to numeric types, turning errors into NaNs
xgboost_df = xgboost_df.apply(pd.to_numeric, errors="coerce")

# Convert the dataframe index to datetime format
xgboost_df.index = pd.to_datetime(xgboost_df.index)

# Arrange the rows chronologically by their datetime index
xgboost_df = xgboost_df.sort_index()

display(xgboost_df.dtypes)
```

IE Wind Forecast (MW)	float64
GB Wind Forecast (MW)	float64
IE Solar Forecast (MW)	float64
GB Solar Forecast (MW)	float64
IE Demand Forecast (MW)	float64
GB Demand Forecast (MW)	float64
EWIC Actual Flow (MW)	float64
Moyle Actual Flow (MW)	float64
Greenlink Actual Flow (MW)	float64
IE DA Prices (€)	float64
GB DA Prices (€)	float64
Net IC Flows (MW)	float64
DA Price Spread (€)	float64
GB Net Position (MW)	float64
IE Net Position (MW)	float64
IE Wind Penetration	float64
GB Wind Penetration	float64
EWIC Capacity (MW)	float64
Greenlink Capacity (MW)	float64
Moyle Capacity (MW)	float64
Total IC Capacity (MW)	float64

dtype: object

Time and Lag Features

Hour, month and day of week are encoded as sine/cosine pairs to preserve continuity across boundaries (e.g. 23:30 into 00:00). A weekend flag is also included.

Lagged flows are added at 24h, 48h and 168h, along with a direction flag for each. Rolling mean, std, max and min are computed over the 24 hours ending at t-24h. The shift is applied before the rolling window to prevent leakage.

Capacity utilisation is the absolute flow as a share of total available capacity, lagged 24 hours.

```
In [6]: # Time features - cyclical encoding preserves continuity across boundaries (e.g. 23:00 to 00:00)
xgboost_df["hour_sin"] = np.sin(2 * np.pi * xgboost_df.index.hour / 24)
xgboost_df["hour_cos"] = np.cos(2 * np.pi * xgboost_df.index.hour / 24)
xgboost_df["month_sin"] = np.sin(2 * np.pi * xgboost_df.index.month / 12)
xgboost_df["month_cos"] = np.cos(2 * np.pi * xgboost_df.index.month / 12)
xgboost_df["dow_sin"] = np.sin(2 * np.pi * xgboost_df.index.dayofweek / 7)
xgboost_df["dow_cos"] = np.cos(2 * np.pi * xgboost_df.index.dayofweek / 7)
xgboost_df["is_weekend"] = (xgboost_df.index.dayofweek >= 5).astype(int)

# Lagged flow features - shift(48)=24h, shift(96)=48h, shift(336)=168h
for lag, label in [(48, "24h"), (96, "48h"), (336, "168h")]:
    xgboost_df[f"Net IC Flow -{label}"] = xgboost_df["Net IC Flows (MW)"].shift(lag)
    xgboost_df[f"Flow Direction -{label}"] = (xgboost_df["Net IC Flows (MW)"].shift(lag) > 0).astype(int)

# Rolling stats over the 24h window ending at t-24h - shift(48) prevents Leakage
xgboost_df["IC Rolling Mean -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).mean()
xgboost_df["IC Rolling Std -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).std()
xgboost_df["IC Rolling Max -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).max()
xgboost_df["IC Rolling Min -24h"] = xgboost_df["Net IC Flows (MW)"].shift(48).rolling(48).min()

# Capacity utilisation - how close flow was to max available capacity, lagged 24h
xgboost_df["Capacity Utilisation -24h"] = (
    xgboost_df["Net IC Flows (MW)"].shift(48).abs() / xgboost_df["Total IC Capacity (MW)"].shift(48)
)

print(f"Date range: {xgboost_df.index[0]} to {xgboost_df.index[-1]}")

# Define the target variable
target = "Net IC Flows (MW)"

# Exclude the target and its components to prevent data Leakage
leak_cols = [
    "Net IC Flows (MW)",
    "EWIC Actual Flow (MW)",
    "Moyle Actual Flow (MW)",
    "Greenlink Actual Flow (MW)",
]

features = [col for col in xgboost_df.columns if col not in leak_cols]

print(f"Target: {target}")
print(f"Features ({len(features)}):")
for f in features:
    print(f"    {f}")
```

Date range: 2025-04-18 23:00:00 to 2026-07-27 00:00:00

Target: Net IC Flows (MW)

Features (35):

IE Wind Forecast (MW)
GB Wind Forecast (MW)
IE Solar Forecast (MW)
GB Solar Forecast (MW)
IE Demand Forecast (MW)
GB Demand Forecast (MW)
IE DA Prices (€)
GB DA Prices (€)
DA Price Spread (€)
GB Net Position (MW)
IE Net Position (MW)
IE Wind Penetration
GB Wind Penetration
EWIC Capacity (MW)
Greenlink Capacity (MW)
Moyle Capacity (MW)
Total IC Capacity (MW)
hour_sin
hour_cos
month_sin
month_cos
dow_sin
dow_cos
is_weekend
Net IC Flow -24h
Flow Direction -24h
Net IC Flow -48h
Flow Direction -48h
Net IC Flow -168h
Flow Direction -168h
IC Rolling Mean -24h
IC Rolling Std -24h
IC Rolling Max -24h
IC Rolling Min -24h
Capacity Utilisation -24h

Train/Test Split

The data is split 80/20 chronologically. No shuffling is applied as the model relies on time-based lag features and random shuffling would cause leakage.

```
In [7]: X = xgboost_df[features].astype(float)
y = xgboost_df[target].astype(float)

split_idx = int(len(X) * 0.8)

X_train = X.iloc[:split_idx]
X_test = X.iloc[split_idx:]
y_train = y.iloc[:split_idx]
y_test = y.iloc[split_idx:]

print("80% train, 20% split")
print(f"Train: {X_train.index[0]} to {X_train.index[-1]} ({len(X_train)} rows)")
print(f"Test: {X_test.index[0]} to {X_test.index[-1]} ({len(X_test)} rows)")
```

80% train, 20% split

Train: 2025-04-18 23:00:00 to 2026-04-24 03:00:00 (17769 rows)

Test: 2026-04-24 03:30:00 to 2026-07-27 00:00:00 (4443 rows)

Hyperparameter Tuning and Model Training

Optuna is used to search for the best hyperparameters over 50 trials, minimising mean RMSE across 5-fold time series cross-validation. Folds are ordered chronologically so no future data leaks into training.

Early stopping is applied within each fold to avoid overfitting. The test set is held out entirely during tuning.

Once the best parameters are found, the model is retrained on the full training set and evaluated on the held-out test set using MAE, RMSE and R^2 .

```
In [8]: optuna.logging.set_verbosity(optuna.logging.WARNING)

# 5-fold time series cross-validation - test set is never seen during tuning
tscv = TimeSeriesSplit(n_splits=5)

def objective(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 200, 2000),           # number of boosting rounds
        "learning_rate": trial.suggest_float("learning_rate", 0.001, 0.1, log=True), # step size shrinkage to reduce overfitting
        "max_depth": trial.suggest_int("max_depth", 3, 8),                   # maximum depth of each tree
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),             # fraction of rows sampled per tree
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0), # fraction of features sampled per tree
        "colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.5, 1.0), # fraction of features sampled per tree level
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 20),    # minimum sum of instance weight in a leaf
        "gamma": trial.suggest_float("gamma", 0.0, 1.0),                   # minimum loss reduction required to split a node
        "reg_alpha": trial.suggest_float("reg_alpha", 0.0, 2.0),             # L1 regularisation on leaf weights
        "reg_lambda": trial.suggest_float("reg_lambda", 0.0, 2.0),          # L2 regularisation on leaf weights
        "random_state": 42,
        "eval_metric": "rmse",
        "n_jobs": -1,
    }

    fold_rmse = []
    for train_idx, val_idx in tscv.split(X_train):
        X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

        m = xgb.XGBRegressor(**params, early_stopping_rounds=50)
        m.fit(X_tr, y_tr, eval_set=[(X_val, y_val)], verbose=False)
```

```

        preds = m.predict(X_val)
        fold_rmse.append(np.sqrt(mean_squared_error(y_val, preds)))

    return np.mean(fold_rmse)

study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=10, show_progress_bar=True)

print(f"Best CV RMSE: {round(study.best_value, 2)} MW")
print(f"Best params: {study.best_params}")

# Retrain on full training set with best hyperparameters
model = xgb.XGBRegressor(
    **study.best_params,
    random_state=42,
    early_stopping_rounds=50,
    eval_metric="rmse",
    n_jobs=-1,
)

model.fit(
    X_train, y_train,
    eval_set=[(X_test, y_test)],
    verbose=100,
)

y_pred = model.predict(X_test)
mae = round(mean_absolute_error(y_test, y_pred), 2)
rmse = round(np.sqrt(mean_squared_error(y_test, y_pred)), 2)
r2 = round(r2_score(y_test, y_pred), 4)

print(f"MAE: {mae} MW")
print(f"RMSE: {rmse} MW")
print(f"R²: {r2}")

```

```

0%|          | 0/10 [00:00<?, ?it/s]
Best CV RMSE: 413.04 MW
Best params: {'n_estimators': 1729, 'learning_rate': 0.006858272283352329, 'max_depth': 4, 'subsample': 0.5485225754416276, 'colsample_bytree': 0.94659874
91918076, 'colsample_bylevel': 0.5968940510937469, 'min_child_weight': 18, 'gamma': 0.8933300963763984, 'reg_alpha': 0.8441918950329723, 'reg_lambda': 0.7
466338144377711}
[0]      validation_0-rmse:483.29565
[100]    validation_0-rmse:395.69905
[200]    validation_0-rmse:361.20149
[300]    validation_0-rmse:346.42233
[400]    validation_0-rmse:340.73005
[500]    validation_0-rmse:337.97475
[600]    validation_0-rmse:337.29888
[700]    validation_0-rmse:336.99663
[745]    validation_0-rmse:337.06760
MAE:   256.15 MW
RMSE:  336.89 MW
R²:    0.5167

```

Feature Importance:

```

In [9]: trained_features = model.get_booster().feature_names

importance = pd.Series(
    model.feature_importances_, index=trained_features
).sort_values(ascending=True)

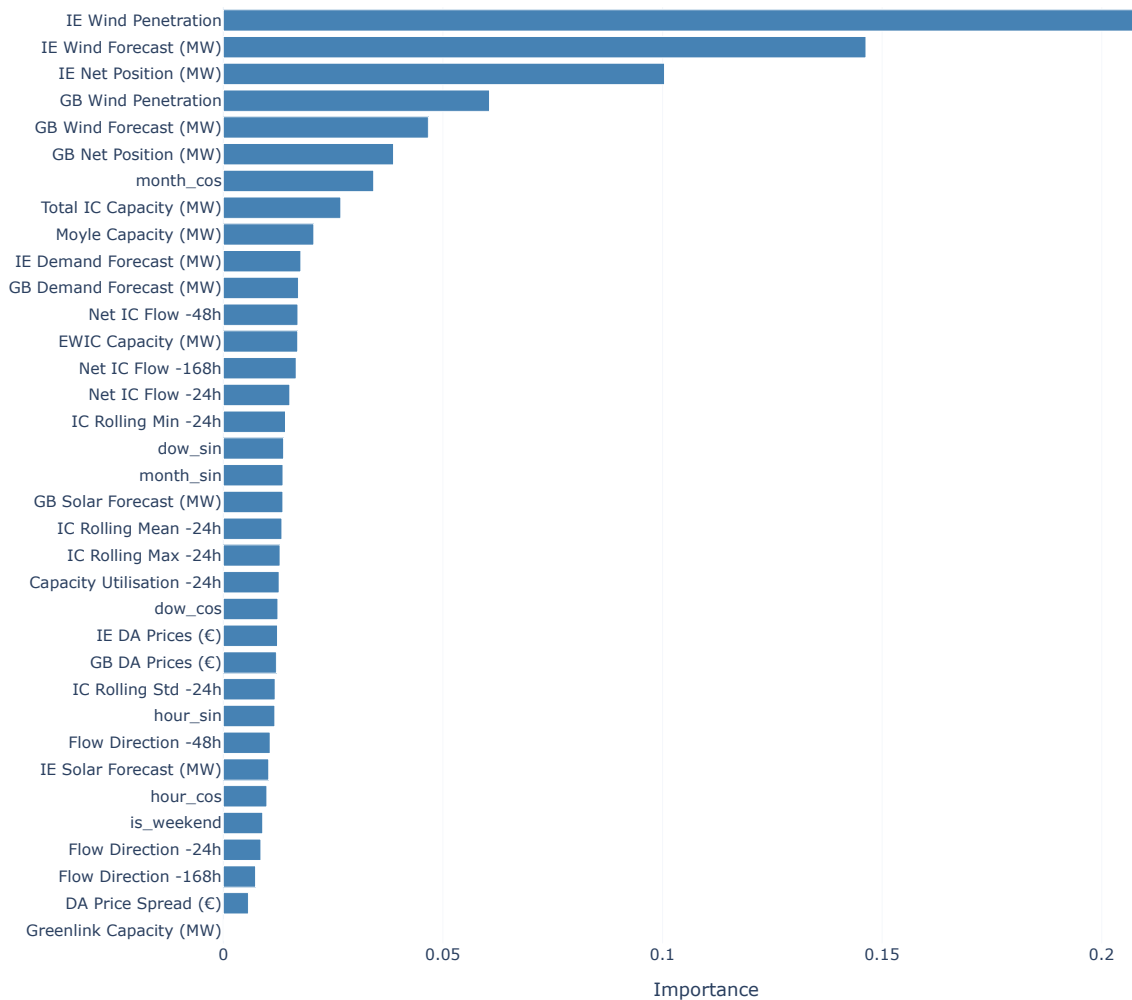
fig2 = go.Figure(go.Bar(
    x=importance.values,
    y=importance.index,
    orientation="h",
    marker_color="steelblue"
))

fig2.update_layout(
    title="XGBoost Feature Importance",
    xaxis_title="Importance",
    template="plotly_white",
    height=900,
    width=1000
)

fig2.show()

```

XGBoost Feature Importance



Model Results:

```
In [10]: now = pd.Timestamp.now()

pred_series = pd.Series(y_pred, index=X_test.index)
upcoming = pred_series[pred_series.index >= now]
upcoming_df = upcoming.reset_index()
upcoming_df.columns = ["Time (UTC)", "Predicted Net IC Flow (MW)"]
upcoming_df["Predicted Net IC Flow (MW)"] = upcoming_df["Predicted Net IC Flow (MW)"].round(1)
upcoming_df["Direction"] = upcoming_df["Predicted Net IC Flow (MW)"].apply(
    lambda x: "IE importing from GB" if x > 0 else "IE exporting to GB"
)
print(f"\nNext predicted IC flows from {now.strftime('%Y-%m-%d %H:%M')} UTC:\n")
display(upcoming_df)

# Plot of the full test set showing Actual vs Forecasted IC Flow
fig1 = go.Figure()

fig1.add_trace(go.Scatter(
    x=X_test.index, y=y_test,
    name="Actual", line=dict(color="black", width=1)
))
fig1.add_trace(go.Scatter(
    x=X_test.index, y=y_pred,
    name="Predicted", line=dict(color="blue", width=1), opacity=0.7
))
fig1.add_hline(y=0, line_dash="dash", line_color="red", opacity=0.5)
fig1.add_vline(
    x=now.timestamp() * 1000,
    line_dash="dot", line_color="grey", opacity=0.8,
    annotation_text="Now", annotation_position="top right"
)
fig1.update_layout(
    title="Actual vs Predicted Net IC Flows - Full Test Set",
    xaxis_title="Time (UTC)",
    yaxis_title="IC Flow (MW)",
    hovermode="x unified",
    template="plotly_white",
    height=600,
    width=1000,
    legend=dict(x=0, y=-0.2, orientation="h")
)
fig1.show()

# Plot of tomorrow's predicted flows
```

```
window_end = X_test.index[-1]
window_start = window_end - pd.Timedelta(days=7)

mask = (X_test.index >= window_start)
x_zoom = X_test.index[mask]
y_actual = y_test[mask]
y_hat = pd.Series(y_pred, index=X_test.index)[mask]

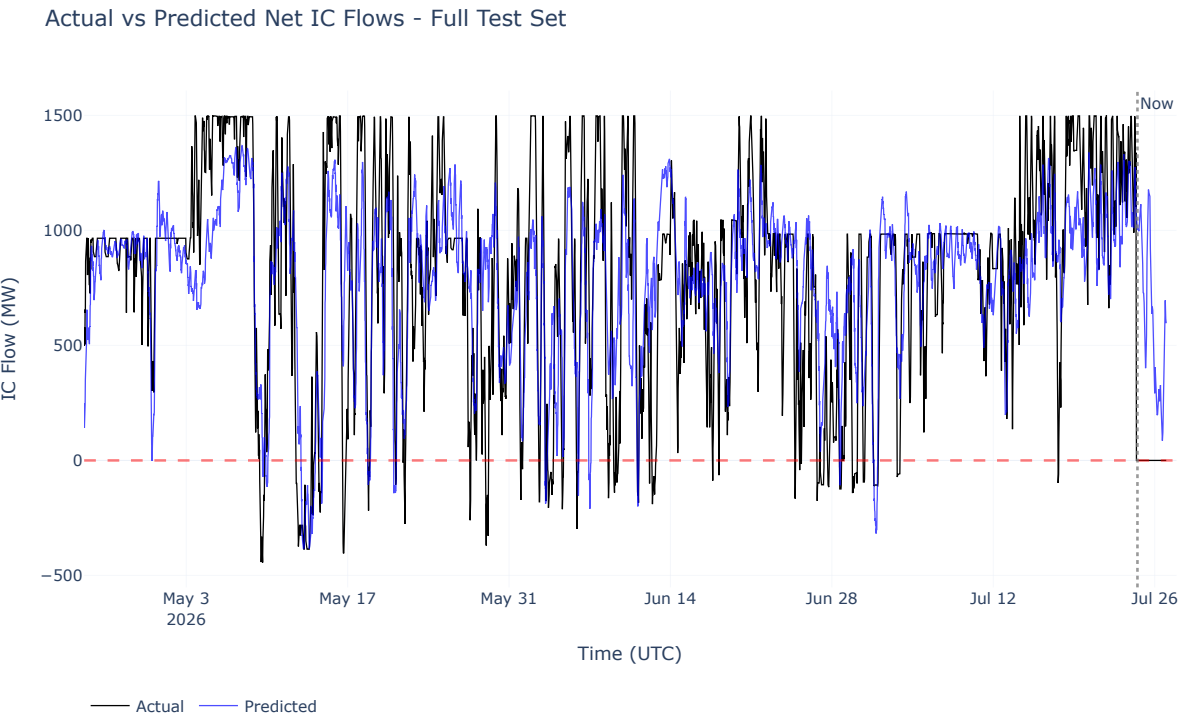
fig2 = go.Figure()

fig2.add_trace(go.Scatter(
    x=x_zoom, y=y_actual,
    name="Actual", line=dict(color="black", width=1.5)
))
fig2.add_trace(go.Scatter(
    x=x_zoom, y=y_hat,
    name="Predicted", line=dict(color="blue", width=1.5), opacity=0.7
))
fig2.add_hline(y=0, line_dash="dash", line_color="red", opacity=0.5)
fig2.add_vline(
    x=now.timestamp() * 1000,
    line_dash="dot", line_color="grey", opacity=0.8,
    annotation_text="Now", annotation_position="top right"
)
fig2.update_layout(
    title="Actual vs Predicted Net IC Flows - Last 7 and Next 2 Days of Test Set",
    xaxis_title="Time (UTC)",
    yaxis_title="IC Flow (MW)",
    hovermode="x_unified",
    template="plotly_white",
    height=600,
    width=1000,
    legend=dict(x=0, y=-0.2, orientation="h")
)
fig2.show()
```

Next predicted IC flows from 2026-07-24 11:05 UTC:

	Time (UTC)	Predicted Net IC Flow (MW)	Direction
0	2026-07-24 12:00:00	1004.299988	IE importing from GB
1	2026-07-24 13:00:00	1004.700012	IE importing from GB
2	2026-07-24 14:00:00	1007.099976	IE importing from GB
3	2026-07-24 15:00:00	996.400024	IE importing from GB
4	2026-07-24 16:00:00	1014.900024	IE importing from GB
...	...	...	...
56	2026-07-26 20:00:00	507.700012	IE importing from GB
57	2026-07-26 21:00:00	684.299988	IE importing from GB
58	2026-07-26 22:00:00	696.200012	IE importing from GB
59	2026-07-26 23:00:00	615.099976	IE importing from GB
60	2026-07-27 00:00:00	596.599976	IE importing from GB

61 rows × 3 columns



Actual vs Predicted Net IC Flows - Last 7 and Next 2 Days of Test Set

